

MARINA

Software Blueprint, SRS & Design System Specification v1.1

Business: Marina Gadgets Kano

Frontend Stack: Pure HTML5, CSS3, Vanilla JS

Status: Engineering Blueprint

Architecture Update (v1.1)

This specification has been updated to standardize the entire frontend architecture on **Pure HTML5, CSS3, and Vanilla JavaScript (ES6+)** with native Django Templates. All third-party dynamic libraries (HTMX, Alpine.js) have been systematically replaced with standard Fetch API calls, native DOM manipulation, and explicit JS module structures.

PART I: SOFTWARE BLUEPRINT V1.1

1. Project Mission

Marina is a premium e-commerce platform built exclusively for **Marina Gadgets Kano**. Its primary purpose is to provide customers with a fast, trustworthy, and modern shopping experience while giving Marina staff powerful tools to manage products, inventory, orders, payments, deliveries, customer communication, and business growth.

Although Marina begins as an e-commerce platform, every architectural decision supports future expansion into additional Marina business lines without requiring a major redesign. The initial release focuses solely on e-commerce.

2. Project Goals

Customer Goals

- Browse products effortlessly with instantaneous native JavaScript search and filtering.
- Purchase securely through integrated, verified payment gateways.
- Track every order in real time.
- Communicate easily with Marina staff via direct channels.
- Build long-term trust through transparent workflows.

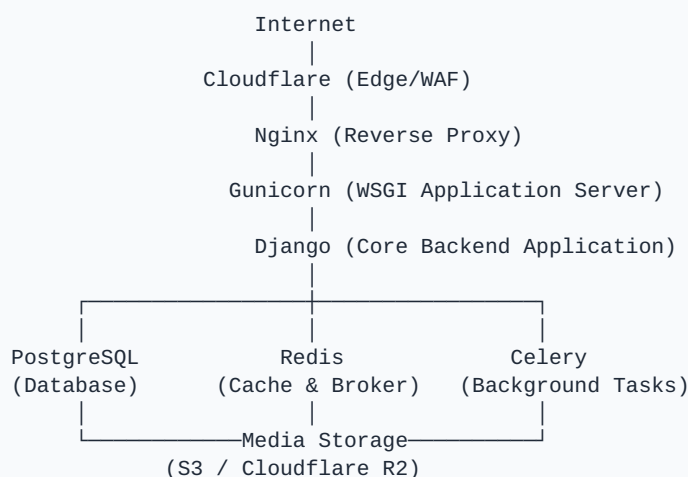
Business Goals

- Increase online sales conversion.
- Reduce manual administrative work.
- Improve real-time inventory accuracy.
- Elevate customer satisfaction.
- Establish reusable infrastructure for future Marina entities.

3. Engineering Philosophy

1. **Simple:** Pure HTML5, modular CSS, native Vanilla JS, simple deployment, and predictable maintenance.
2. **Scalable:** Support a scaling lifecycle from 10 → 100 → 1,000 → 100,000 active users without architectural redesign.
3. **Maintainable:** Zero external JS framework dependencies; fast onboarding for new engineers through pure standards-compliant web technologies.
4. **Secure:** Built-in security defaults across database, sessions, CSRF token validation, and REST endpoints.
5. **Performance:** Fast initial page loads, lightweight native scripts, optimized assets, intelligent caching, and lazy loading.

4. High-Level Architecture



5. Technology Stack

- **Backend:** Django (Python) — Secure, high-performance ORM, built-in admin, robust security defaults.
- **API Framework:** Django REST Framework (DRF) — Exposes REST endpoints for asynchronous AJAX requests and future mobile applications.
- **Database:** PostgreSQL — Enterprise-grade reliability, ACID compliance, native JSONB indexing.
- **Cache & Message Broker:** Redis — OTP session management, response caching, rate limiting, and Celery broker services.
- **Task Queue:** Celery — Asynchronous workers for OTP delivery, WhatsApp/SMS messaging, image optimization, and scheduled jobs.
- **Web Server & Proxy:** Gunicorn + Nginx
- **Media Storage:** S3-compatible Object Storage (Cloudflare R2 / AWS S3 / Backblaze B2)
- **Frontend: Standard HTML5 + CSS3 + Vanilla JavaScript (ES6+)**
 - *Reason:* Zero framework bloat, standard web compliance, complete control over DOM updates, native `fetch()` API for AJAX requests, lightweight performance, and high long-term maintainability.

6. Folder Structure

```
marina/
├── config/                # Core settings, URLs, WSGI/ASGI
├── apps/
│   ├── accounts/         # Auth & identity
│   ├── customers/        # Profiles & preferences
│   ├── catalog/          # Products, categories
│   ├── brands/           # Brand management
│   ├── inventory/        # Stock tracking & audit logs
│   ├── cart/             # Cart logic
│   ├── wishlist/         # Customer wishlists
│   ├── checkout/         # Checkout workflows
│   ├── orders/           # Order fulfillment & status
│   ├── payments/         # Gateway integrations
│   ├── delivery/         # Logistics & dispatch tracking
│   ├── notifications/    # SMS, WhatsApp, In-App alerts
│   ├── rewards/          # Loyalty & store credits
│   ├── feedback/         # "Build Marina With Us"
│   ├── media/            # Media asset processing
│   ├── homepage/         # Banners & dynamic sections
│   ├── analytics/        # Business intelligence
│   ├── dashboard/        # Custom staff portal UI
│   └── core/             # Utilities & shared mixins
├── templates/            # Server-rendered HTML5 templates
├── static/
│   ├── css/              # Custom modular CSS stylesheets
│   ├── js/               # Pure Vanilla JavaScript modules (main.js, cart.js, etc.)
│   └── images/           # Static visual assets
├── media/                # Local development media storage
└── requirements/         # Python dependency manifests
```

7. User Types & RBAC Matrix

Role	Permissions Summary
Visitor	Browse public catalog, search items, view specifications, read static content.
Customer	OTP login, manage profile/addresses, cart management, wishlist, checkout, track orders, feedback.
Sales Staff	Review orders, confirm offline orders, update order status, communicate with buyers.
Inventory Staff	Product creation/editing, upload media, brand/category management, stock updates.
Dispatch Staff	Assign carriers, record departure times, upload package verification photos, update tracking.
Customer Support	Lookup customer records, manage support requests, respond to feedback.
Marketing Staff	Manage homepage banners, featured products, promotional announcements.
Administrator	Full business operations management, staff account setup, analytics access.
Super Admin	System configuration, low-level database operations, developer logs, emergency maintenance.

PART II: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

V1.1

SECTION 1 — System Overview & Business Rules

Core Business Rules (BR)

- **BR-001:** Every customer account is uniquely identified by a verified phone number.
- **BR-002:** Passwords are not used for customer accounts. Authentication relies solely on OTP verification.
- **BR-003:** Authentication requires One-Time Password (OTP) verification unless originating from an active trusted device session.
- **BR-004:** Customers cannot purchase out-of-stock items.
- **BR-005:** Inventory deduction occurs automatically upon online payment confirmation or staff confirmation for offline checkouts.
- **BR-006:** Every stock adjustment must produce an immutable audit log entry.
- **BR-007:** Orders cannot be permanently purged from the database; they may only be archived or cancelled.
- **BR-008:** Every payment transaction must maintain a unique, deterministic reference string.
- **BR-009:** Guest checkout is not supported. Unregistered users completing checkout are transparently registered via inline phone OTP verification.
- **BR-010:** Catalog price changes do not affect historical orders; line items preserve purchase-time prices.

SECTION 2 — Functional Requirements

FR-01: Authentication Engine (Pure JS + Django API)

The login interface uses custom Vanilla JavaScript to submit AJAX phone verification requests without triggering full-page reloads.

```
// Vanilla JS Authentication Handler Example
document.getElementById('otp-form')?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const phone = document.getElementById('phone-input').value;
  const csrfToken = document.querySelector('[name=csrfmiddlewaretoken]').value;

  const response = await fetch('/api/v1/auth/send-otp/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRFToken': csrfToken
    },
    body: JSON.stringify({ phone: phone })
  });
  const data = await response.json();
  if (response.ok) {
    document.getElementById('step-phone').style.display = 'none';
    document.getElementById('step-otp').style.display = 'block';
  } else {
    showToast(data.message || 'Error sending code', 'danger');
  }
});
```

FR-02: Shopping Cart & Dynamic DOM Updates

All shopping cart operations (quantity updates, item removals, price updates) are handled asynchronously via standard Fetch API calls. JavaScript directly modifies DOM node content and updates header cart badge counts in real time.

FR-03: Search & Filter Module

Product filters send asynchronous requests to Django REST endpoints. The returned HTML snippet or JSON payload is dynamically rendered inside the catalog grid using native JavaScript DOM manipulation methods.

FR-04: Checkout & Payment Integrations

- **Online Payment:** Direct API communication with OPay SDK / REST API.
- **WhatsApp Assisted Checkout:** Native JavaScript formats cart parameters into a pre-filled `https://wa.me/` URL and redirects the client.
- **Request a Call:** Form submits AJAX payload to trigger a staff notification call queue.

PART III & IV: UI/UX & DESIGN SYSTEM (MDS V1.1)

1. Color System

Token	Hex Code	Applied Purpose
--color-primary	#185ABD	Primary Buttons, Active Navigation, Key Accents
--color-primary-hover	#144A99	Hover / Active states for primary controls
--color-primary-light	#EAF3FF	Background highlights, active card tints
--color-success	#16A34A	In-Stock badges, Payment confirmed, Success toasts
--color-warning	#F59E0B	Low Stock alerts, Pending verification warnings
--color-danger	#DC2626	Out of Stock badges, Destructive actions, Error toasts
--color-bg	#FFFFFF	Primary page canvas background
--color-surface	#F8F AFC	Secondary backgrounds, Cards, Table zebra rows
--color-text-primary	#1F2937	Primary body text, Headings
--color-text-secondary	#6B7280	Subtitles, Labels, Captions

2. Vanilla CSS Styling Architecture (Sample)

```
/* Standard MDS CSS Custom Properties */
:root {
  --color-primary: #185ABD;
  --color-primary-hover: #144A99;
  --color-bg: #FFFFFF;
  --color-surface: #F8FAFC;
  --color-text-primary: #1F2937;
  --radius-button: 12px;
  --radius-card: 16px;
  --shadow-card: 0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03);
  --font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
}

/* Reusable Button System */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 10px 20px;
  border-radius: var(--radius-button);
  font-family: var(--font-family);
  font-weight: 600;
  font-size: 14px;
  border: none;
  cursor: pointer;
  transition: background-color 150ms ease, transform 150ms ease;
}

.btn-primary {
  background-color: var(--color-primary);
  color: #FFFFFF;
}

.btn-primary:hover {
  background-color: var(--color-primary-hover);
}

.btn-primary:active {
  transform: scale(0.98);
}
```

3. Brand Voice & UI Interaction Matrix

Action / Context	Incorrect / Generic	Standard Marina Copy (Human & Clear)
Form Action	"Submit"	"Send Feedback" / "Save Address"
Checkout CTA	"Checkout"	"Complete Your Order"
OTP Fail	"OTP Invalid"	"That code doesn't look right. Please check it and try again."
Empty Wishlist	"No items found"	"You haven't saved any items yet. Browse our gadgets to save your favorites!"